

## **SYLLABUS UNIT-I**

**Introduction:** Alphabets, Strings, and Languages; Automata and Grammars, Deterministic finite Automata (DFA)-Formal Definition, Simplified notation: State transition graph, Transition table, Language of DFA, Nondeterministic finite Automata (NFA), NFA with epsilon transition, Language of NFA, Equivalence of NFA and DFA, Minimization of Finite Automata, Distinguishing one string from other, Myhill-Nerode Theorem

## Introduction to Theory of Computation

Automata theory, also known as the Theory of Computation, is a field within computer science and mathematics that focuses on studying abstract machines to understand the capabilities and limitations of computation by analyzing mathematical models of how machines can perform calculations.

Now, let's understand the basic terminologies, which are important and frequently used in the Theory of Computation.

### 1. Symbol

A symbol (often also called a **character**) is the smallest building block, which can be any alphabet, letter, or picture.

**a, b, c, 0, 1, .....**

### 2. Alphabets ( $\Sigma$ )

A finite, non-empty set of symbols used to construct strings and languages. For example,  $\Sigma = \{a, b\}$ .

Examples:

$\Sigma = \{0, 1\}$  is an alphabet of binary digits

$\Sigma = \{0, 1, \dots, 9\}$  is an alphabet of decimal digits

$\Sigma = \{a, b, c\}$

$\Sigma = \{A, B, C, \dots, Z\}$

### 3. String

A string is a **finite** sequence of symbols from some alphabet. A string is generally denoted as **w** and the length of a string is denoted as **|w|**. Empty string is the string with zero occurrence of symbols, represented as  $\epsilon$ .

**Number of Strings (of length 2)**

**that can be generated over the alphabet {a, b}:**

--

a a

a b

b a

*b b*

*Length of String  $|w| = 2$*

*Number of Strings = 4*

**Conclusion:**

*For alphabet  $\{a, b\}$  with length  $n$ , number of strings can be generated =  $2n$ .*

## Alphabets:

***n the theory of computation, an alphabet is defined as a finite set of symbols used to construct strings and languages.***

**Definition of Alphabet**

An **alphabet** (denoted as  $\Sigma$ ) is a **finite, non-empty set of symbols**. These symbols can be letters, digits, or other characters. For example:

- **Binary Alphabet:**  $\Sigma = \{0, 1\}$
- **Lowercase Letters:**  $\Sigma = \{a, b, c, \dots, z\}$
- **Uppercase Letters:**  $\Sigma = \{A, B, C, \dots, Z\}$
- **Special Characters:**  $\Sigma = \{+, \&, \%, \dots\}$ .

### **Language**

- A **language** is a set of strings formed using the symbols of a given alphabet  $\Sigma$ .
- Formally, a language is a subset of  $\Sigma^*$ , where  $\Sigma^*$  is the set of all possible strings (including  $\epsilon$ ) over the alphabet  $\Sigma$ .

**Examples of Languages:**

*Finite Language:*

$L1 = \{ \text{set of string of 2} \}$

$L1 = \{ xy, yx, xx, yy \}$

*Infinite Language:*

$L1 = \{ \text{set of all strings starts with 'b'} \}$

$L1 = \{ babb, baa, ba, bbb, baab, \dots \}$

### **Types of Languages in TOC**

Languages are classified based on the computational model or grammar generating them:

1. **Regular Languages:** Defined using regular expressions or finite automata.  
Example:  $L = \{ a^n / n \geq 0 \}$ .
2. **Context-Free Languages:** Defined using context-free grammars or pushdown automata. Example:  $L = \{ a^n b^n / n \geq 0 \}$ .
3. **Context-Sensitive Languages:** Defined using context-sensitive grammars or linear-bounded automata.
4. **Recursive and Recursively Enumerable Languages:** Defined using Turing machines.

## Introduction of Finite Automata

Finite automata are abstract machines used to recognize patterns in input sequences, forming the basis for understanding regular languages in computer science.

- Consist of states, transitions, and input symbols, processing each symbol step-by-step.
- If ends in an accepting state after processing the input, then the input is accepted; otherwise, rejected.
- Finite automata come in deterministic (DFA) and non-deterministic (NFA), both of which can recognize the same set of regular languages.
- Widely used in text processing, compilers, and network protocols.

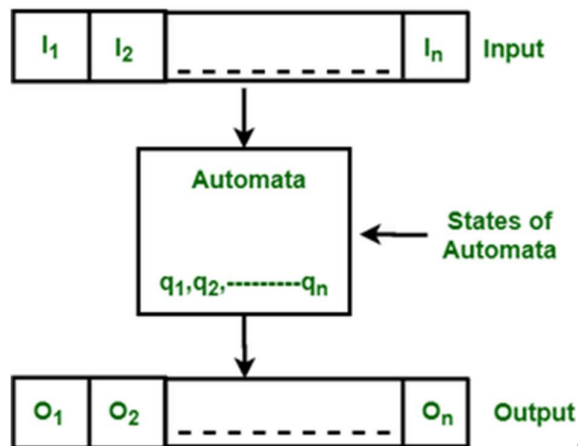


Figure: Features of Finite Automata

## Features of Finite Automata

- **Input:** Set of symbols or characters provided to the machine.
- **Output:** Accept or reject based on the input pattern.
- **States of Automata:** The conditions or configurations of the machine.
- **State Relation:** The transitions between states.
- **Output Relation:** Based on the final state, the output decision is made.

## Formal Definition of Finite Automata

A finite automaton can be defined as a tuple:

$\{Q, \Sigma, q, F, \delta\}$ , where:

- $Q$ : Finite set of states

- $\Sigma$ : Set of input symbols
- $q$ : Initial state
- $F$ : Set of final states
- $\delta$ : Transition function

### Types of Finite Automata

There are two types of finite automata:

- Deterministic Finite Automata (DFA)
- Non-Deterministic Finite Automata (NFA)

#### 1. Deterministic Finite Automata (DFA)

A DFA is represented as  $\{Q, \Sigma, q, F, \delta\}$ . In DFA, for each input symbol, the machine transitions to one and only one state. DFA does not allow any null transitions, meaning every state must have a transition defined for every input symbol.

DFA consists of 5 tuples  $\{Q, \Sigma, q, F, \delta\}$ .

$Q$  : set of all states.

$\Sigma$  : set of input symbols. ( Symbols which machine takes as input )

$q$  : Initial state. ( Starting state of a machine )

$F$  : set of final state.

$\delta$  : Transition Function, defined as  $\delta : Q \times \Sigma \rightarrow Q$ .

#### Example:

Construct a DFA that accepts all strings ending with 'a'.

Given:

$\Sigma = \{a, b\}$ ,

$Q = \{q_0, q_1\}$ ,

$F = \{q_1\}$

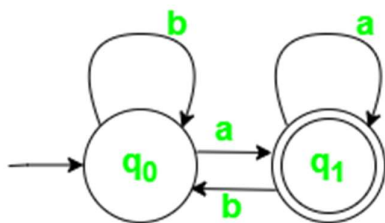


Fig 1. State Transition Diagram for DFA with  $\Sigma = \{a, b\}$

| State\Symbol | a  | b  |
|--------------|----|----|
| q0           | q1 | q0 |
| q1           | q1 | q0 |

In this example, if the string ends in 'a', the machine reaches state q1, which is an accepting state.

## 2) Non-Deterministic Finite Automata (NFA)

NFA is similar to DFA but includes the following features:

- It can transition to multiple states for the same input.
- It allows null ( $\epsilon$ ) moves, where the machine can change states without consuming any input.

Example:

Construct an NFA that accepts strings ending in 'a'.

Given:

$\Sigma = \{a, b\}$ ,

$Q = \{q0, q1\}$ ,

$F = \{q1\}$

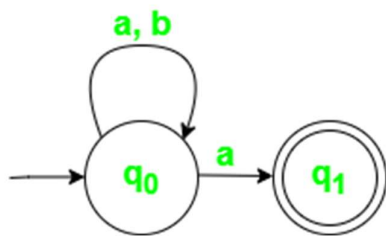


Fig 2. State Transition Diagram for NFA with  $\Sigma = \{a, b\}$

State Transition Table for above Automaton,

| State\Symbol | a       | b  |
|--------------|---------|----|
| q0           | {q0,q1} | q0 |

| <b>State\Symbol</b> | <b><i>a</i></b> | <b><i>b</i></b> |
|---------------------|-----------------|-----------------|
| <i>q1</i>           | $\phi$          | $\phi$          |

*In an NFA, if any transition leads to an accepting state, the string is accepted.*

***Transition graphs***

*Transition graph can be interpreted as a flowchart for an algorithm recognizing a language. A transition graph consists of three things:*

- 1. A finite set of states, at least one of which is designated the start state and some of which are designated as final states.*
- 2. An alphabet  $\Sigma$  of possible input symbols from which the input strings are formed.*
- 3. A finite set of transitions that show the change of state from the given state on a given input.*

*A successful path through the transition graph is a series of edges forming a path beginning at the start state and ending at one of the final states.*

## **Difference between DFA and NFA**

In automata theory four types of finite automata are used to recognize the regular language among these two are DFA and NFA. Both have the same function but there are some differences in their structure and working. It is very essential for one to have a clear understanding of the difference between DFA and NFA for anyone to understand the computational models or to know how to design an algorithm. This article will try to discuss the differences as well as give examples that can help in the understanding of the topics.

### **What is DFA?**

DFA refers to [Deterministic Finite Automaton](#). A Finite Automata(FA) is said to be deterministic if corresponding to an input symbol, there is a single resultant state i.e. there is only one transition.

**A deterministic finite automaton is set of five tuples represented as,**

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

**Where,**

- **Q:** A non-empty finite set of states in the finite control( $q_0, q_1, q_2, \dots$ ).
- **$\Sigma$ :** A non-empty finite set of input symbols.
- **$\delta$ :** It is a transition function that takes two arguments, a state, and an input symbol, it returns a single state.
- **$q_0$ :** It is starting state, one of the states in Q.
- **F:** It is a non-empty set of final states/ accepting states from the set belonging to Q.

### **What is NFA?**

NFA refers to [Nondeterministic Finite Automaton](#). A Finite Automata(FA) is said to be non-deterministic if there is more than one possible transition from one state on the same input symbol.

**A non-deterministic finite automata is also a set of five tuples and represented as,**

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

**Where,**

- **Q:** A set of non empty finite states.

- $\Sigma$ : A set of non empty finite input symbols.
- $\delta$ : It is a transition function that takes a state from  $Q$  and an input symbol from  $\Sigma$  and returns a subset of  $Q$ .
- $q_0$ : Initial state of NFA and member of  $Q$ .
- $F$ : A non-empty set of final states and member of  $Q$ .

#### **Difference Between DFA and NFA**

| <b>DFA</b>                                                                                                 | <b>NFA</b>                                                                               |
|------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <i>DFA stands for Deterministic Finite Automata.</i>                                                       | <i>NFA stands for Nondeterministic Finite Automata.</i>                                  |
| <i>For each symbolic representation of the alphabet, there is only one state transition in DFA.</i>        | <i>No need to specify how does the NFA react according to some symbol.</i>               |
| <i>DFA cannot use Empty String transition.</i>                                                             | <i>NFA can use Empty String transition.</i>                                              |
| <i>DFA can be understood as one machine.</i>                                                               | <i>NFA can be understood as multiple little machines computing at the same time.</i>     |
| <i>In DFA, the next possible state is distinctly set.</i>                                                  | <i>In NFA, each pair of state and input symbol can have many possible next states.</i>   |
| <i>DFA is more difficult to construct.</i>                                                                 | <i>NFA is easier to construct.</i>                                                       |
| <i>DFA rejects the string in case it terminates in a state that is different from the accepting state.</i> | <i>NFA rejects the string in the event of all branches dying or refusing the string.</i> |

| <b>DFA</b>                                                                                                                                   | <b>NFA</b>                                                                                                                                               |
|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>Time needed for executing an input string is less.</i>                                                                                    | <i>Time needed for executing an input string is more.</i>                                                                                                |
| <i>All DFA are NFA.</i>                                                                                                                      | <i>Not all NFA are DFA.</i>                                                                                                                              |
| <i>DFA requires more space.</i>                                                                                                              | <i>NFA requires less space than DFA.</i>                                                                                                                 |
| <i>Dead configuration is not allowed.</i><br><br><i>eg: if we give input as 0 on q0 state so we must give 1 as input to q0 as self loop.</i> | <i>Dead configuration is allowed.</i><br><br><i>eg: if we give input as 0 on q0 state so we can give next input 1 on q1 which will go to next state.</i> |
| <i><math>\delta: Q \times \Sigma \rightarrow Q</math> i.e. next possible state belongs to <math>Q</math>.</i>                                | <i><math>\delta: Q \times (\Sigma \cup \epsilon) \rightarrow 2^Q</math> i.e. next possible state belongs to power set of <math>Q</math>.</i>             |
| <i>Conversion of Regular expression to DFA is difficult.</i>                                                                                 | <i>Conversion of Regular expression to NFA is simpler compared to DFA.</i>                                                                               |
| <i>Epsilon move is not allowed in DFA</i>                                                                                                    | <i>Epsilon move is allowed in NFA</i>                                                                                                                    |
| <i>In a DFA, there is only <b>one</b> possible transition for each input symbol from any given state.</i>                                    | <i>In an NFA, there can be <b>multiple</b> transitions for a single input symbol from a given state.</i>                                                 |

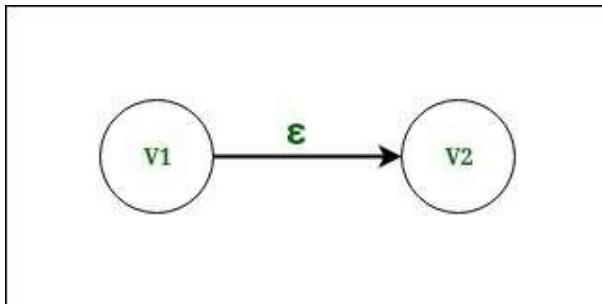
### **Conclusion**

*In conclusion, DFAs are deterministic, uncomplicated to implement but they create greater automata while NFAs are flexible and can be less bulky but difficult to implement. In fact, despite these differences, NFAs and DFAs are as powerful as each other, in that they are able to recognize the same classes of languages. It is possible to convert any NFA to a DFA even though the amount of states in a resultant DFA can be more than in the initial NFA.*

## Conversion of Epsilon-NFA to NFA

An NFA (Non-deterministic Finite Automaton) is a theoretical model used to recognize patterns and determine whether a string belongs to a specific language. An Epsilon-NFA ( $\epsilon$ -NFA) is a special type of NFA where transitions can happen without reading any input symbol, using epsilon ( $\epsilon$ ) transitions.

- An NFA allows multiple possible transitions from a given state based on the input symbol.
- The key difference between them is that Epsilon-NFAs have epsilon transitions, while regular NFAs only transition on specific input symbols.
- Converting an Epsilon-NFA to an NFA eliminates the epsilon transitions, allowing us to apply standard NFA algorithms for language recognition.



Epsilon ( $\epsilon$ ) Transition

## Epsilon-NFA to NFA Conversion Steps

### Step 1: Find the Epsilon Closure

- For each state in the Epsilon-NFA, find its epsilon closure.
- This means figuring out all the states that can be reached by only using epsilon transitions (and include the state itself).
- This helps in tracking which states can be reached without reading any input symbol.

### Step 2: Create New States for the NFA

- Each state in the new NFA corresponds to a set of states you found in the epsilon closures.
- The starting state in the NFA will be the epsilon closure of the initial state of the Epsilon-NFA.

### Step 3: Define the Transitions

- For each state in the NFA (which represents a group of states from the epsilon closures), look at what happens when you read each input symbol.

- For an input symbol  $a$ , check which states you can reach from any state in the current group, and then find the epsilon closure of those states.
- This way, you're considering both direct transitions and the ones that happen because of epsilon moves.

#### Step 4: Set Accepting States

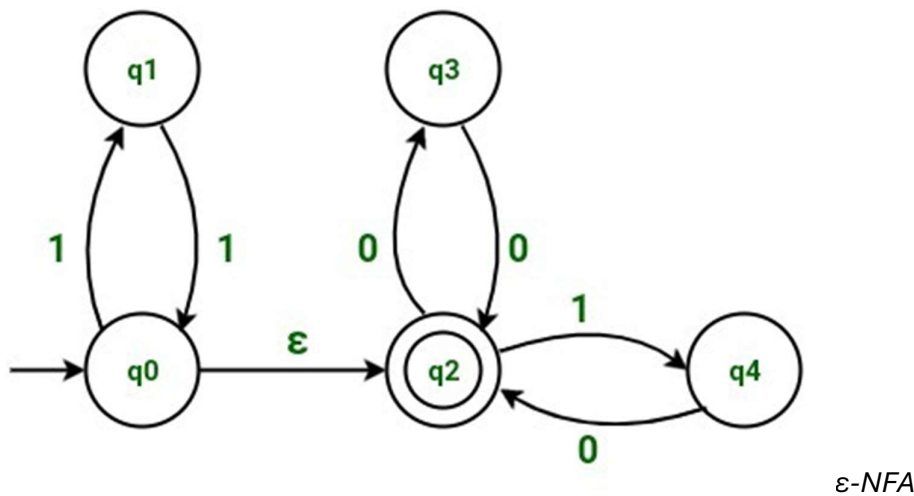
- Any state in the NFA that includes at least one accepting state from the Epsilon-NFA becomes an accepting state.
- This ensures that the NFA recognizes all the same strings as the Epsilon-NFA.

#### Step 5: Keep Going Until Done

- Keep processing each new state and its transitions until no more new states are created.
- This guarantees that every possible state is covered, and there are no epsilon transitions left.

#### Example of Epsilon-NFA to NFA Conversion

Convert the following  $\epsilon$ -NFA to NFA. The  $\epsilon$ -NFA has states  $q_0, q_1, q_2, q_3, q_4$  where  $q_0$  is the initial state and  $q_2$  final state.



#### Solution

In the above example, we have 5 states named as  $q_0, q_1, q_2, q_3$  and  $q_4$ . Initially, we have  $q_0$  as start state and  $q_2$  as final state. We have  $q_1, q_3$  and  $q_4$  as intermediate states.

According to  $\epsilon$ -NFA the states follow following transitions:

- State  $q_0$  when,

- **Input 0:** No transition
- **Input 1:** Moves to  $q_1$
- **Input  $\epsilon$ :** Moves to  $q_2$
- State  $q_1$  when,
  - **Input 0:** No transition
  - **Input 1:** Moves to  $q_0$
  - **Input  $\epsilon$ :** No transition
- State  $q_2$  when,
  - **Input 0:** Moves to  $q_3$
  - **Input 1:** Moves to  $q_4$
  - **Input  $\epsilon$ :** No transition
- State  $q_3$  when,
  - **Input 0:** Moves to  $q_2$
  - **Input 1:** No transition
  - **Input  $\epsilon$ :** No transition
- State  $q_4$  when,
  - **Input 0:** Moves to  $q_2$
  - **Input 1:** No transition
  - **Input  $\epsilon$ :** No transition

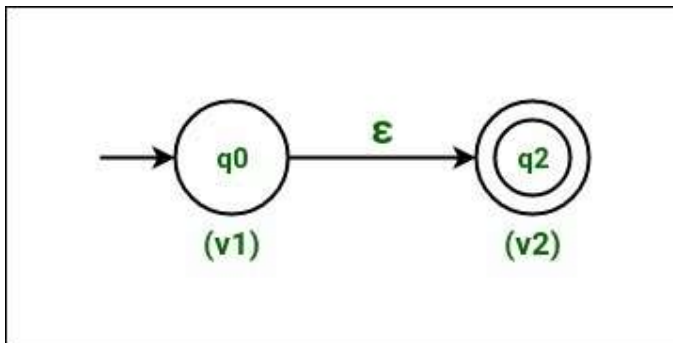
**Transition table for the state transitions:**

| States/Input | Input 0 | Input 1 | Input epsilon |
|--------------|---------|---------|---------------|
| $q_0$        | -       | $q_1$   | $q_2$         |
| $q_1$        | -       | $q_0$   | -             |
| $q_2$        | $q_3$   | $q_4$   | -             |

| States/Input | Input 0 | Input 1 | Input epsilon |
|--------------|---------|---------|---------------|
| q3           | q2      | -       | -             |
| q4           | q2      | -       | -             |

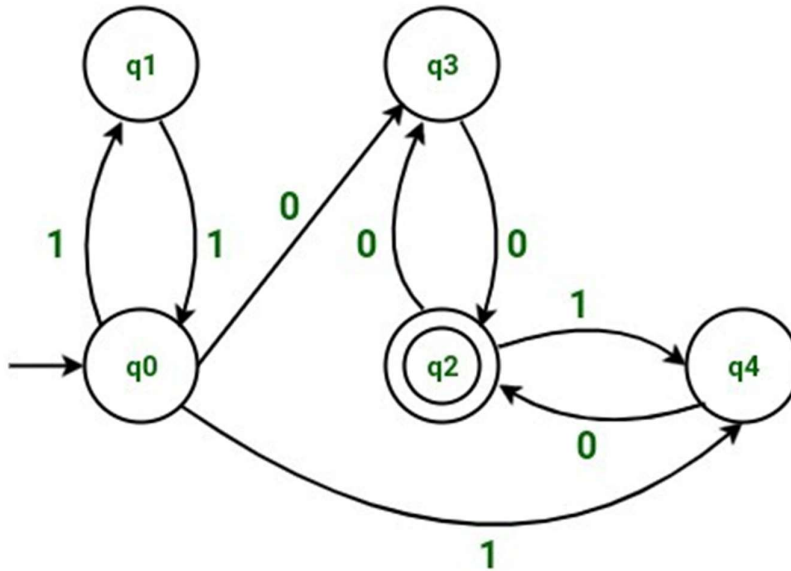
We can see that we have an epsilon move from state  $q_0$  to state  $q_2$ , which is to be removed. To remove epsilon move from state  $q_0$  to state  $q_1$ , we will follow the steps mentioned below.

**Step 1:** Considering the epsilon move from state  $q_0$  to state  $q_2$ . Consider the state  $q_0$  as vertex  $v_1$  and state  $q_2$  as vertex  $v_2$ .



**Step 2:** Now find all the moves that starts from vertex  $v_2$  (i.e. state  $q_2$ ). After finding the moves, duplicate all the moves that start from vertex  $v_2$  (i.e. state  $q_2$ ) with the same input to start from vertex  $v_1$  (i.e. state  $q_0$ ) and remove the epsilon move from vertex  $v_1$  (i.e. state  $q_0$ ) to vertex  $v_2$  (i.e. state  $q_2$ ). Since state  $q_2$  on getting input 0 goes to state  $q_3$ . Hence on duplicating the move, we will have state  $q_0$  on getting input 0 also to go to state  $q_3$ .

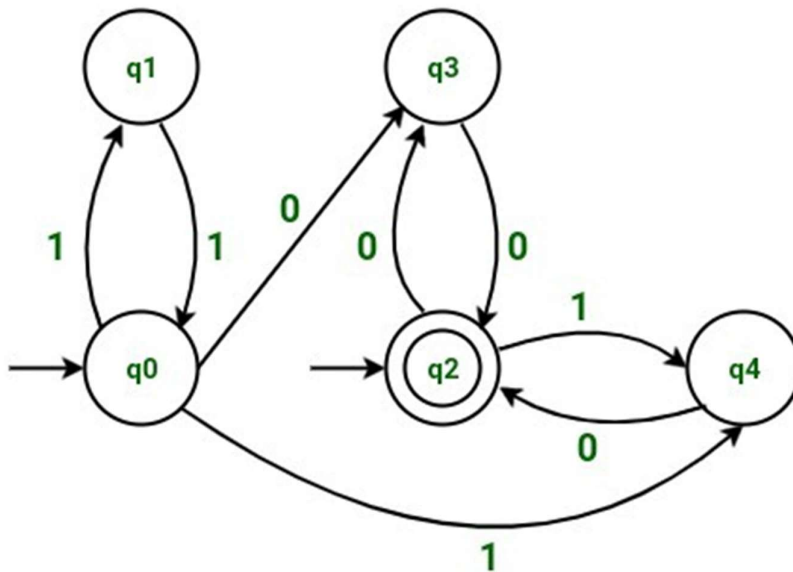
Similarly state  $q_2$  on getting input 1 goes to state  $q_4$ . Hence on duplicating the move, we will have state  $q_0$  on getting input 1 also to go to state  $q_4$ . So, NFA after duplicating the moves is:



NFA on Duplicating

Moves

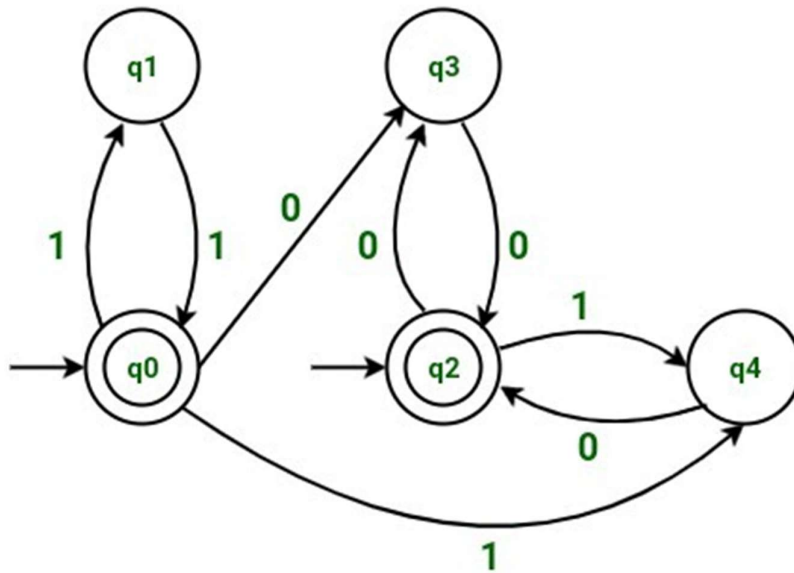
**Step 3:** Since vertex  $v_1$  (i.e. state  $q_0$ ) is a start state. Hence we will also make vertex  $v_2$  (i.e. state  $q_2$ ) as a start state. Note that state  $q_2$  will also remain as a final state as we had initially. NFA after making state  $q_2$  also as a start state is:



NFA after making

state  $q_2$  as a start state

**Step 4:** Since vertex  $v_2$  (i.e. state  $q_2$ ) is a final state. Hence we will also make vertex  $v_1$  (i.e. state  $q_0$ ) as a final state. Note that state  $q_0$  will also remain as a start state as we had initially. After making state  $q_0$  also as a final state, the resulting NFA is:



Resulting NFA (state q0

as a final state)

The transition table for the above resulting NFA is:

| States/Input | Input 0 | Input 1 |
|--------------|---------|---------|
| q0           | q3      | q1,q4   |
| q1           | -       | q0      |
| q2           | q3      | q4      |
| q3           | q2      | -       |
| q4           | q2      | -       |

The transition graph in the figure represents a DFA with at least one a and at least one b.



## **Core Areas of the Theory of Computation**

*The field of computation theory can be broadly divided into three major areas:*

### **1. Automata Theory**

*Automata theory studies abstract computational models and their applications. It forms the basis for understanding how machines process inputs and produce outputs. Key components include:*

- **Finite Automata:** *Used to model simple systems like lexical analyzers in compilers.*
- **Pushdown Automata:** *A more powerful model capable of recognizing context-free languages, essential for parsing programming languages.*
- **Turing Machines:** *The most powerful automata, used as a standard for defining what is computable.*

### **2. Formal Languages and Grammars**

*This area examines the syntax and structure of languages used in computation. It involves:*

- **Regular Languages:** *Described by regular expressions and finite automata, representing simple patterns.*
- **Context-Free Languages:** *Defined by context-free grammars, crucial for designing compilers.*
- **Chomsky Hierarchy:** *A classification of languages into regular, context-free, context-sensitive, and recursively enumerable languages.*

### **3. Computability and Decidability**

*Computability theory addresses the question: What problems can a computer solve? It studies concepts like:*

- **Decidable Problems:** *Problems with an algorithmic solution.*
- **Undecidable Problems:** *Problems, such as the Halting Problem, for which no algorithm can determine a solution for all inputs.*

### **4. Complexity Theory**

*Complexity theory focuses on the efficiency of algorithms by analyzing the time and space resources they require. It categorizes problems into classes such as:*

- **P (Polynomial Time):** *Problems solvable in polynomial time.*

- **NP (Nondeterministic Polynomial Time):** Problems whose solutions can be verified in polynomial time.
- **NP-Complete and NP-Hard:** The most challenging problems in NP, with applications in cryptography, optimization, and artificial intelligence.

## **Grammar in Theory of Computation**

*In Theory of Computation, grammar refers to a formal system that defines how strings in a language are constructed. It plays a crucial role in determining the syntactic correctness of languages and forms the foundation for parsing and interpreting programming languages, natural languages, and other formal systems.*

*This article provides an in-depth exploration of:*

- *The types of grammars.*
- *Their components.*
- *The process of string derivation using grammar rules.*

## **Grammar in Computation**

**Grammar** is a formal system that defines a set of rules for generating valid strings within a language. It serves as a blueprint for constructing syntactically correct sentences or meaningful sequences in a formal language.

Grammar is basically composed of two basic elements:

- **Terminal Symbols:** Terminal symbols are those that are the components of the sentences generated using grammar and are represented using small case letters like a, b, c, etc.
- **Non-Terminal Symbols:** Non-terminal symbols are those symbols that take part in the generation of the sentence but are not the component of the sentence. Non-Terminal Symbols are also called Auxiliary Symbols and Variables. These symbols are represented using a capital letters like A, B, C, etc.

## **Representation of Grammar**

Any Grammar can be represented by 4 tuples -  $\langle N, T, P, S \rangle$

- **N** - Finite Non-Empty Set of Non-Terminal Symbols.
- **T** - Finite Set of Terminal Symbols.
- **P** - Finite Non-Empty Set of Production Rules.

- **S - Start Symbol** (Symbol from where we start producing our sentences or strings).

### **Production Rules**

A production or production rule in computer science is a rewrite rule specifying a symbol substitution that can be recursively performed to generate new symbol sequences. It is of the form  $\alpha \rightarrow \beta$  where  $\alpha$  is a Non-Terminal Symbol which can be replaced by  $\beta$  which is a string of Terminal Symbols or Non-Terminal Symbols.

### **Example 1**

**Consider Grammar  $G1 = \langle N, T, P, S \rangle$**

$T = \{a, b\}$  #Set of terminal symbols

1 2 3 4 5

$P = \{A \rightarrow Aa, A \rightarrow Ab, A \rightarrow a, A \rightarrow b, A \rightarrow \epsilon\}$  #Set of all production rules

$S = \{A\}$  #Start Symbol

As the start symbol  $S$  is equivalent to  $A$  then we can produce  $Aa, Ab, a, b, \epsilon$  strings. These strings can further produce strings where  $A$  can be replaced by the strings mentioned in the production rules. Hence this grammar can be used to produce strings of the form  $(a+b)^*$ .

### **Derivation of Strings**

$A \rightarrow a$  #using production rule 3

**OR**

$A \rightarrow Aa$  #using production rule 1

$Aa \rightarrow ba$  #using production rule 4

**OR**

$A \rightarrow Aa$  #using production rule 1

$Aa \rightarrow AAa$  #using production rule 1

$AAa \rightarrow bAa$  #using production rule 4

$bAa \rightarrow ba$  #using production rule 5

### **Example 2**

**Consider Grammar  $G2 = \langle N, T, P, S \rangle$**

$N = \{A\}$  #Set of non-terminals Symbols

$T = \{a\}$  #Set of terminal symbols

1 2 3 4

$P = \{A \rightarrow Aa, A \rightarrow AAa, A \rightarrow a, A \rightarrow \epsilon\}$  #Set of all production rules

$S = \{A\}$  #Start Symbol

As the start symbol is S then we can produce Aa, AAa, a, which can further produce strings where A can be replaced by the Strings mentioned in the production rules and hence this grammar can be used to produce strings of form  $(a)^*$ .

### **Derivation of Strings**

A  $\rightarrow$  a #using production rule 3

**OR**

A  $\rightarrow$  Aa #using production rule 1

Aa  $\rightarrow$  aa #using production rule 3

**OR**

A  $\rightarrow$  Aa #using production rule 1

Aa  $\rightarrow$  AAa #using production rule 1

AAa  $\rightarrow$  Aa #using production rule 4

Aa  $\rightarrow$  aa #using production rule 3

### **Equivalent Grammars**

Two grammars are said to be equivalent if they generate the same language. For instance, if Grammar 1 and Grammar 2 both generate strings of the form  $(a^+b)^*$ , they are considered equivalent.

### **Types of Grammars**

There are several types of Grammar. We classify them on the basis mentioned below.

- **Type of Production Rules:** The form and complexity of the production rules define the grammar type, such as context-free, context-sensitive, regular, or unrestricted grammars.
- **Number of Derivation Trees:** The number of ways a string can be derived from the grammar. Ambiguous grammars have multiple derivation trees for the same string.
- **Number of Strings:** The size and nature of the language generated by the grammar.

## Language of NFA (Non-deterministic Finite Automaton)

### 1. Basic Idea

- An **NFA** (Non-deterministic Finite Automaton) is a finite automaton where:
    - For a given state and input symbol, **multiple transitions** are possible (including none).
    - It can also have  **$\epsilon$ -moves (epsilon transitions)**, meaning the automaton can change states without consuming any input symbol.
  - The **language of an NFA** refers to the **set of all strings that the NFA accepts**.
- 

### 2. Formal Definition

An NFA is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

- $Q \rightarrow$  Finite set of states.
- $\Sigma \rightarrow$  Input alphabet.
- $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$   
 $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q \rightarrow$  Transition function (maps a state and symbol to a set of states).
- $q_0 \in Q$   
 $q_0 \in Q \rightarrow$  Start state.
- $F \subseteq Q$   
 $F \subseteq Q \rightarrow$  Set of accepting (final) states.

#### 👉 Language of NFA $L(M)$ :

$L(M) = \{w \in \Sigma^* \mid \text{there exists at least one path from } q_0 \text{ to some } f \in F \text{ on input } w\}$   
 $L(M) = \{w \in \Sigma^* \mid \text{there exists at least one path from } q_0 \text{ to some } f \in F \text{ on input } w\}$

---

### 3. Acceptance Condition

A string **w** is accepted by an NFA if:

1. Starting from initial state  $q_0$ ,

2. There exists **at least one computation path** (due to nondeterminism) that consumes all symbols of  $w$ ,
3. And ends in an **accepting state (final state)**.

⚡ Unlike DFA, where there is **only one path** for each input, in NFA there may be **many possible paths**, but **only one accepting path is enough**.

---

#### 4. Example

Consider NFA  $M$ :

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- $q_0$  is the start state
- $F = \{q_2\}$
- Transition function  $\delta$ :
  - $\delta(q_0, 0) = \{q_0, q_1\}$
  - $\delta(q_1, 1) = \{q_2\}$

**Language:**

- Strings where **a 0 is followed by a 1** (because  $q_0 \rightarrow q_1 \rightarrow q_2$ ),
- Plus any number of 0s before that (because  $q_0$  loops on 0).
- So,

$$L(M) = \{0^n 1 \mid n \geq 1\}$$


---

#### 5. Properties of NFA Languages

- **Equivalence with DFA:**  
Every NFA has an equivalent DFA that accepts the same language.  
(But DFA may have exponentially more states.)
- **Closure Properties:**  
Languages accepted by NFAs (regular languages) are **closed under**:
  - Union
  - Intersection

- Concatenation
  - Kleene star ( $^{**}$ )
  - Complementation
  - **NFA = DFA in Power:**  
NFAs do not accept “extra” languages; they just provide a **more flexible description**.
- 

## 6. Point-to-Point Summary

-  NFA is a 5-tuple  $(Q, \Sigma, \delta, q_0, F)$ .
-  Language of NFA = Set of strings accepted by at least one path ending in a final state.
-  Acceptance needs only **one successful path** (not all).
-  NFA may have **multiple transitions** or  **$\epsilon$ -moves**.
-  NFAs and DFAs are equivalent in expressive power (both accept **regular languages**).
-  NFAs are often easier to construct than DFAs.

## Equivalence of NFA and DFA

| Feature                                 | NFA (Non-deterministic Finite Automaton)                                                  | DFA (Deterministic Finite Automaton)                                 |
|-----------------------------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------|
| Transitions                             | For a given state and input, can move to <b>multiple states</b> (0, 1, or more).          | For a given state and input, must move to <b>exactly one state</b> . |
| $\epsilon$ -moves (epsilon transitions) | Allowed (state change without consuming input).                                           | Not allowed.                                                         |
| Computation paths                       | Multiple computation paths possible for one input string.                                 | Only one unique computation path for each input string.              |
| Acceptance condition                    | A string is accepted if <b>at least one path</b> ends in a final state.                   | A string is accepted if the <b>only path</b> ends in a final state.  |
| Ease of construction                    | Easier to construct for complex languages (fewer states).                                 | Sometimes harder to construct, usually more states required.         |
| Determinism                             | Non-deterministic $\rightarrow$ machine may "guess" next move.                            | Deterministic $\rightarrow$ next move is uniquely decided.           |
| Conversion                              | Every NFA can be converted to an equivalent DFA using <b>subset construction method</b> . | DFA is already deterministic; no need for conversion.                |
| Language power                          | Accepts <b>Regular Languages</b> .                                                        | Accepts <b>Regular Languages</b> .                                   |
| Efficiency                              | Requires less effort to design but harder to implement directly.                          | Easier to implement in machines but may require more states.         |

---

### Key Point of Equivalence

- **NFA and DFA accept the same class of languages: Regular Languages.**
- Difference is only in **representation** and **state transitions**, not in **language power**.

## Minimization of Finite Automata

### 1. Meaning

- **Minimization of Finite Automata** means **reducing the number of states** in a DFA (Deterministic Finite Automaton) without changing the language it accepts.
  - The minimized DFA is the **smallest possible DFA** for a given regular language.
  - Important because:
    - Less memory needed.
    - Faster computation.
    - Unique up to isomorphism (for a given regular language, minimized DFA is unique).
- 

### 2. Why Minimization is Needed

- When converting **NFA**  $\rightarrow$  **DFA**, or from a **regular expression**  $\rightarrow$  **DFA**, the DFA often has **extra/unnecessary states**.
  - Minimization helps to:
    - Remove **unreachable states**.
    - Merge **equivalent states**.
    - Produce the **canonical DFA** (smallest version).
- 

### 3. Steps of DFA Minimization

There are **two main methods**:

1. **Table-filling (Equivalence) Method**
  2. **Partition (State Equivalence) Method**
- 

#### Step-by-Step (Partition Method – Commonly Used)

##### Step 1: Remove Unreachable States

- Identify all states that cannot be reached from the start state.
  - Delete them (they are useless).
-

### **Step 2: Initial Partition**

- Divide states into **two groups**:
    - Final states (accepting states).
    - Non-final states.
- 
- 

### **Step 3: Refine the Partition**

- Iteratively split groups into smaller groups based on their transitions.
  - Two states are **equivalent** if for every input symbol, their transitions go to the **same group**.
  - Otherwise, split them into different groups.
- 
- 

### **Step 4: Repeat Until Stable**

- Keep refining partitions until no further splitting is possible.
- 
- 

### **Step 5: Construct the Minimized DFA**

- Each partition/group becomes a **single state** in the minimized DFA.
  - Transitions between groups are defined accordingly.
- 
- 

## **4. Example**

Suppose DFA has:

- States:  $\{A, B, C, D, E\}$
- Start:  $A$
- Final:  $\{C, E\}$
- Alphabet:  $\{0, 1\}$

### **Minimization Steps:**

1. Remove unreachable states  $\rightarrow$  (assume all reachable).
2. Partition:  $\{\{C, E\}, \{A, B, D\}\}$
3. Refine: Split  $\{A, B, D\}$  if they behave differently on inputs.

- *If A and B go to different groups on input 0 or 1, they must split.*
  - *Repeat until stable.*
4. *Final groups = Minimized DFA states.*
- 

## **5. Properties of Minimized DFA**

- *Unique (no two different minimized DFAs exist for the same language, except renaming of states).*
  - *Has the **minimum number of states** possible.*
  - *Useful in **compiler design, lexical analysis, and pattern matching**.*
- 

## **6. Point-to-Point Summary**

- ☒ **Definition:** *Process of reducing states in DFA without changing the language.*
- ☒ **Step 1:** *Remove unreachable states.*
- ☒ **Step 2:** *Partition into final & non-final states.*
- ☒ **Step 3:** *Refine partitions until stable.*
- ☒ **Step 4:** *Construct minimized DFA.*
- ☒ **Result:** *Unique smallest DFA.*

## Basic Theorems in TOC (Myhill-Nerode Theorem)

The Myhill-Nerode theorem is a fundamental result in the theory of formal languages and automata. It helps in determining whether a given language is regular (i.e., whether it can be recognized by a finite automaton) or not. This theory was proven by **John Myhill** and **Anil Nerode** in 1958.

Before understanding the Myhill-Nerode theorem, it's essential to know about formal languages, regular languages, finite automata, and equivalence classes of strings.

1. **Language:** A set of strings (like words or sequences of symbols) over some alphabet (a set of symbols).
2. **Regular Language:** A language is regular if it can be recognized by a finite automaton, which is like a machine that checks if a string belongs to the language.

### Myhill-Nerode Theorem

The Myhill-Nerode Theorem states:

*A language  $L$  over an alphabet  $\Sigma^*$  is regular if and only if there are finitely many equivalence classes under the equivalence relation  $\sim_L$ , where for any two strings  $x$  and  $y$ , we say  $x \sim_L y$  if for every string  $z$ , the concatenation  $xz \in L$  if and only if  $yz \in L$ .*

Now, let's define the equivalence relation that arises from this theorem.

### Equivalence Relation

From the above idea, we define a relation  $\sim_L$ :  $x \sim_L y$  means strings  $x$  and  $y$  are indistinguishable by language  $L$ .

This relation is an equivalence relation, meaning:

- **Reflexive:**  $x \sim_L x$  (a string is always indistinguishable from itself).
- **Symmetric:** If  $x \sim_L y$ , then  $y \sim_L x$ .
- **Transitive:** If  $x \sim_L y$  and  $y \sim_L z$ , then  $x \sim_L z$ .

Thus, this [equivalence relation](#) splits all possible strings into equivalence classes, grouping indistinguishable strings together.

**Important note:** Each equivalence class contains either strings all belonging to  $L$ , or all not belonging to  $L$ .

### Equivalence Class

An [equivalence class](#) is a group of strings that behave the same way when checked by an automaton for a given language  $L$ . Two strings  $x$  and  $y$  are in the same equivalence class if, no matter what you add to them, they will either both be accepted by the automaton or both rejected.

**For example:**

*Strings like "01" and "001" would be in the same equivalence class because adding the same string to both of them results in either both being accepted or both being rejected.*

Now, let's look at the concept of distinguishable strings.

**Distinguishable Strings**

For a language  $L \subseteq \Sigma^*$ , two strings  $x$  and  $y$  are said to be distinguishable if there exists another string  $z$ , called a distinguishing extension, that makes exactly one of  $xz$  or  $yz$  belong to the language  $L$ .

- If no such  $z$  exists, the strings are called indistinguishable.

**Example:**

- Suppose  $L$  is a language of all strings ending with '01'.
- Strings "0" and "1" are distinguishable because the extension "1" gives "01" (in  $L$ ) and "11" (not in  $L$ ).

**Application of Myhill-Nerode Theorem**

The Myhill-Nerode theorem has several key applications:

**1. Proving Non-Regularity:** It helps to prove that a language is not regular by showing that the number of equivalence classes is infinite.

**2. Minimizing DFAs:** It aids in minimizing [deterministic finite automata](#) (DFAs) by identifying distinct equivalence classes, which correspond to states in the minimal DFA.

Read more about [Minimization of DFA](#)

**3. Characterizing Regular Languages:** It provides a way to determine if a language is regular by checking if the number of equivalence classes is finite.

**4. Solving Decision Problems:** It can be used to solve problems like checking if two DFAs accept the same language.

**5. Language Recognition:** It helps construct automata for regular languages by analyzing equivalence classes.

**6. Simplifying Automata Theory Proofs:** It offers a more straightforward method for proving properties of [regular languages](#), avoiding complex constructions.